COGS 109 Final Project: Predicting Sleep Quality from Wearable Multisensor Data

Team Members: Anthony Mitine, Harshatha Prasanna, Yuxing Liu

Introduction

Wearable devices provide a non-invasive way to monitor sleep and daily activity, but transforming raw physiological and movement data into meaningful sleep quality measures remains a challenge. In this project, we use the MMASH dataset, which includes participant sleep records, actigraphy data, heart rate, beat-to-beat interval data, activity logs, questionnaire scores, and demographic information.

Our cleaned analysis dataset contains **n = 21 observations** after excluding one participant with incomplete data, and we use **p = 6 predictors**: age, BMI, average heart rate, total steps, mean vector magnitude, and heavy activity events. Our goal is to predict a continuous sleep quality score from wearable and behavioral features. Since the dataset does not include clinical sleep staging like REM duration, our sleep quality score is derived from available sleep-related variables such as total sleep time, efficiency, wake after sleep onset, number of awakenings, and sleep fragmentation index. We also examine whether daytime activity level is associated with sleep quality.

Hypothesis

Hypothesis: Participants with higher daytime physical activity (higher `total_steps`, higher `mean_vector_mag`, and more `heavy_activity_events`) will have higher Sleep Quality Scores.

Null hypothesis: Daytime physical activity is not associated with the Sleep Quality Score.

This hypothesis ties directly to the features used in modeling and will be evaluated using cross-validated prediction error and the direction of the fitted coefficients.

In [1]:
```
# Import libraries
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression, Lasso, LassoCV
from sklearn.model_selection import KFold, cross_val_score, cross_val_predict, Grid
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.dummy import DummyRegressor
```

```
import statsmodels.api as sm
import statsmodels.formula.api as smf

# Set plot style
sns.set_theme(style="whitegrid")
```

Data Wrangling & Aggregation

Iterating through the `data/user_*` directories to extract and aggregate daytime features and sleep target variables for each of the 22 users.

In [2]:
```
base_path = 'data'
compiled_data = []

# Loop through all 22 user folders
for i in range(1, 23):
    user_folder = f"user_{i}"
    user_path = os.path.join(base_path, user_folder)

    if not os.path.exists(user_path):
        continue

    user_dict = {'user_id': i}

    # Load User Info
    try:
        user_info = pd.read_csv(os.path.join(user_path, 'user_info.csv'))
        user_dict['age'] = user_info['Age'].values[0]
        user_dict['gender'] = user_info['Gender'].values[0]
        user_dict['bmi'] = user_info['Weight'].values[0] / ((user_info['Height'].va
    except:
        pass

    # Load Sleep Data
    try:
        sleep_df = pd.read_csv(os.path.join(user_path, 'sleep.csv'))
        user_dict['total_sleep_time'] = sleep_df['Total Sleep Time (TST)'].values[0
        user_dict['latency_efficiency'] = sleep_df['Efficiency'].values[0]
        user_dict['waso'] = sleep_df['Wake After Sleep Onset (WASO)'].values[0]
        user_dict['awakenings'] = sleep_df['Number of Awakenings'].values[0]
        user_dict['sleep_frag_index'] = sleep_df['Sleep Fragmentation Index'].value
    except:
        pass

    # Aggregate Actigraph Data
    try:
        act_df = pd.read_csv(os.path.join(user_path, 'Actigraph.csv'))
        # Average daytime HR and total steps
        user_dict['avg_hr'] = act_df['HR'].mean()
        user_dict['total_steps'] = act_df['Steps'].sum()
        user_dict['mean_vector_mag'] = act_df['Vector Magnitude'].mean()
    except:
        pass
```

```
    # Aggregate Activity Log
    try:
        activity_df = pd.read_csv(os.path.join(user_path, 'Activity.csv'))
        heavy_acts = activity_df[activity_df['Activity'] == 6]
        user_dict['heavy_activity_events'] = len(heavy_acts)
    except:
        pass

    compiled_data.append(user_dict)

# Create dataframe
df = pd.DataFrame(compiled_data)
df.head()
```

Out[2]:

	user_id	age	gender	bmi	total_sleep_time	latency_efficiency	waso	awakenings
0	1	29	M	22.758307	144.0	87.27	21.0	9.0
1	2	27	M	28.367524	244.0	73.49	84.0	18.0
2	3	34	M	23.120624	351.0	79.23	89.0	16.0
3	4	27	M	23.456790	319.0	85.52	50.0	28.0
4	5	25	M	20.824656	348.0	85.71	58.0	21.0

Data Cleaning

Creating the continuous target variable: **Sleep Quality Score (0 - 100)**. We will normalize positive factors (TST, Efficiency) and penalize them using normalized disruption factors (WASO, Awakenings, Fragmentation).

In [3]:

```
# Drop any users with missing vital data to ensure clean modeling
df = df.dropna().copy()

# Min-Max scale components between 0 and 1 for equal weighting
def min_max_scale(series):
    return (series - series.min()) / (series.max() - series.min())

# Positive factors
scaled_tst = min_max_scale(df['total_sleep_time'])
scaled_eff = min_max_scale(df['latency_efficiency'])

# Negative factors (
inv_waso = 1 - min_max_scale(df['waso'])
inv_awake = 1 - min_max_scale(df['awakenings'])
inv_frag = 1 - min_max_scale(df['sleep_frag_index'])

# Composite Score Equation
# Weighting: 40% TST, 20% Efficiency, 15% WASO, 15% Awakenings, 10% Fragmentation
df['sleep_quality_score'] = ((scaled_tst * 40) + (scaled_eff * 20) + (inv_waso * 15
```

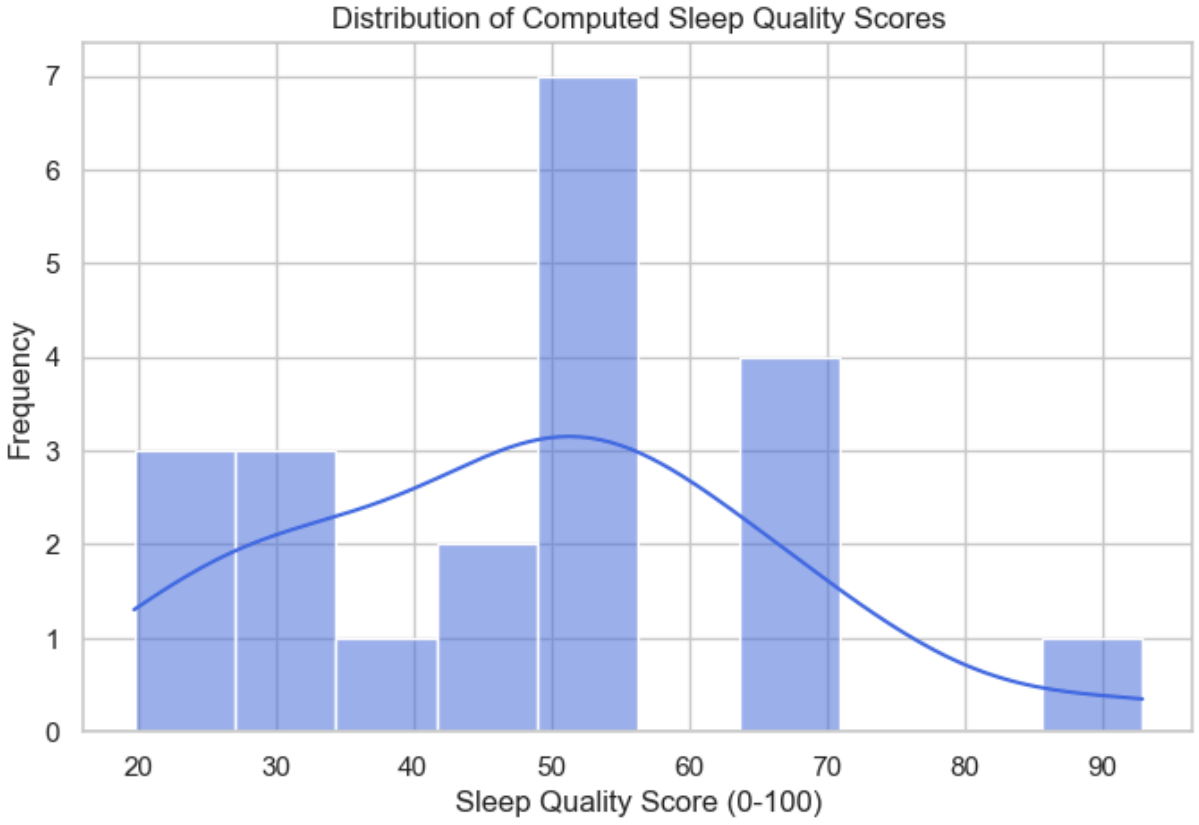
```
# Display the distribution of our new target variable
df[['user_id', 'sleep_quality_score']].head()
```

Out[3]:

	user_id	sleep_quality_score
0	1	48.227881
1	2	30.645749
2	3	46.445878
3	4	50.017850
4	5	54.785845

Exploratory Data Analysis (EDA)

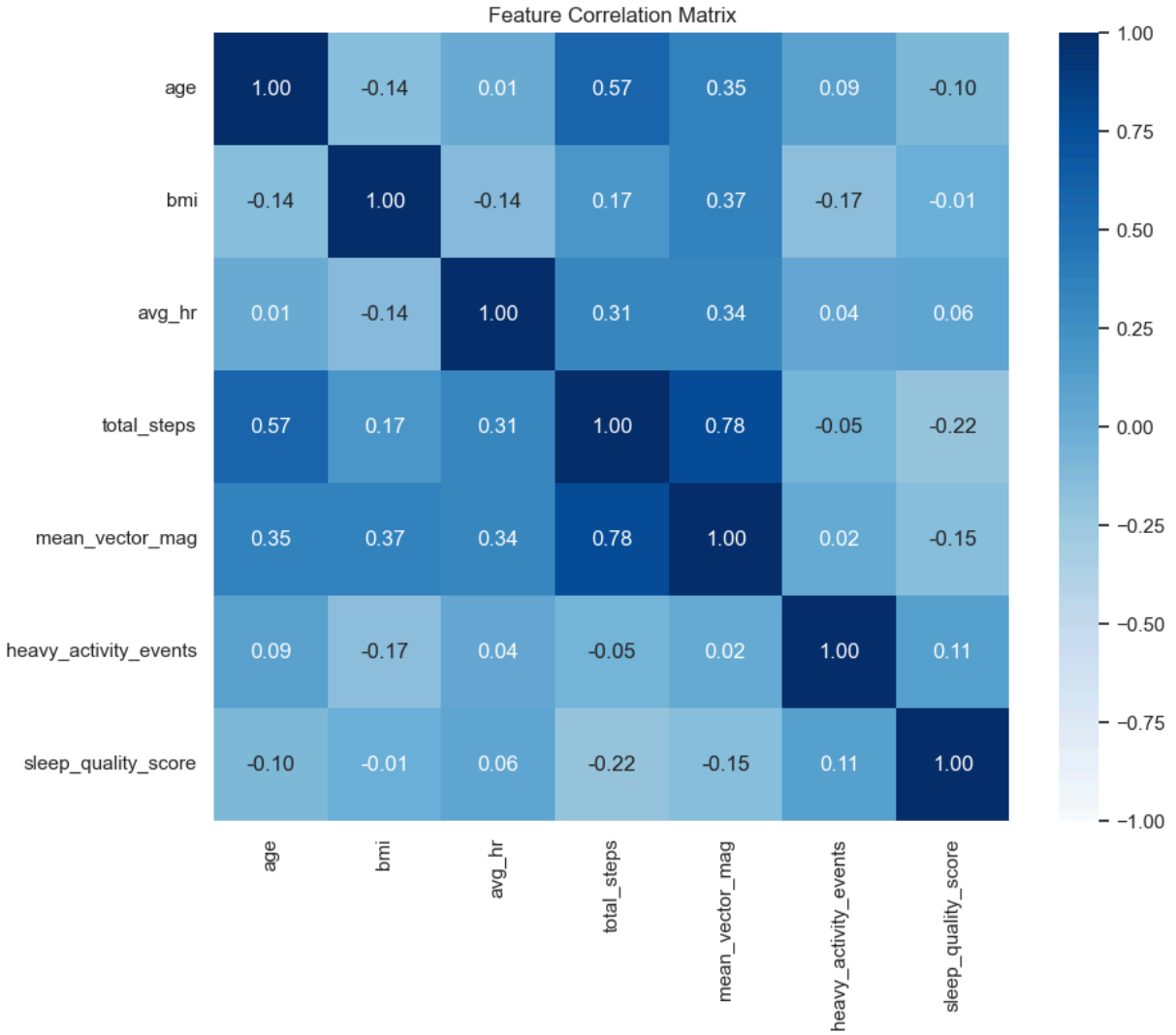
In [4]:
```
# Distribution of the Engineered Target Variable
plt.figure(figsize=(8, 5))
sns.histplot(df['sleep_quality_score'], bins=10, kde=True, color='royalblue')
plt.title('Distribution of Computed Sleep Quality Scores')
plt.xlabel('Sleep Quality Score (0-100)')
plt.ylabel('Frequency')
plt.show()
```



The computed Sleep Quality Scores form a roughly normal distribution centered around 50, with a wide spread of values ranging from the low 20s up to the 90s. This variance is ideal for

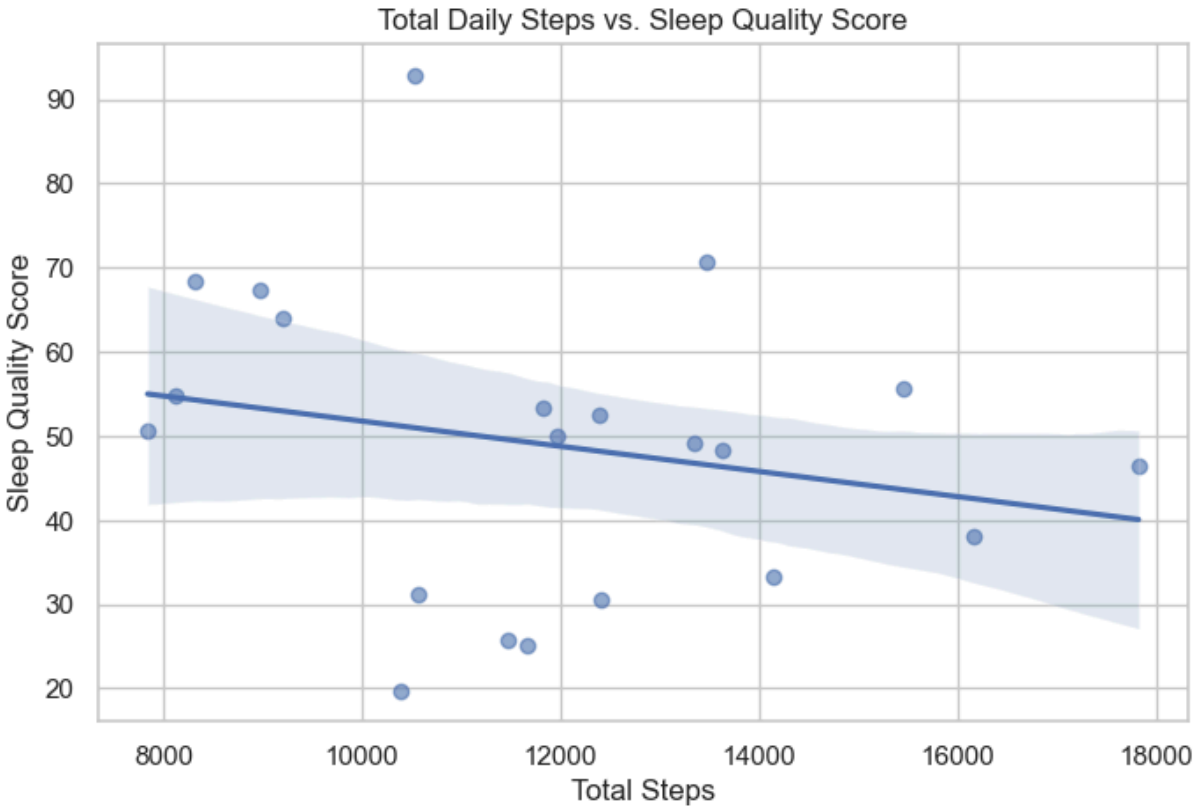
our regression task; if the scores were highly clustered, our model would struggle to identify meaningful patterns.

In [5]:
```
# Correlation Heatmap of Predictive Features vs Target
features = ['age', 'bmi', 'avg_hr', 'total_steps', 'mean_vector_mag', 'heavy_activi
corr_matrix = df[features].corr()

plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, cmap='Blues', fmt=".2f", vmin=-1, vmax=1)
plt.title('Feature Correlation Matrix')
plt.show()
```



The heatmap reveals a strong correlation (0.78) between `total_steps` and `mean_vector_mag`. This indicates high multicollinearity, meaning these two features provide redundant information about a user's daytime movement. This perfectly justifies our methodological choice to implement Lasso Regression, which will automatically penalize and shrink redundant coefficients. Additionally, we see that individual daytime features generally have weak linear correlations with the target variable (e.g., `total_steps` has a -0.22 correlation), suggesting that a multiple regression model combining several features will be necessary to achieve predictive accuracy.

In [6]:
```
# Visualizing a potential relationship: Total Steps vs Sleep Quality
plt.figure(figsize=(8, 5))
sns.regplot(data=df, x='total_steps', y='sleep_quality_score', scatter_kws={'alpha'
plt.title('Total Daily Steps vs. Sleep Quality Score')
plt.xlabel('Total Steps')
plt.ylabel('Sleep Quality Score')
plt.show()
```



The scatter plot visualizes the slightly negative trend between daytime step count and our engineered sleep score. While counterintuitive, this highlights the complexity of physiological data, simply walking more does not automatically guarantee better sleep in this specific sample, further emphasizing the need for a multivariate approach.

Data Preprocessing

Before model estimation, we must separate our predictors (X) from our target (y) and standardize the features. Standardization ensures that variables with vastly different scales (e.g., `total_steps` vs `heavy_activity_events`) are treated equally by the Lasso regularization penalty.

In [7]:
```
# Define features (X) and target (y)
feature_cols = ['age', 'bmi', 'avg_hr', 'total_steps', 'mean_vector_mag', 'heavy_ac
X = df[feature_cols]
y = df['sleep_quality_score']

# The final model comparison uses cross-validation with scaling handled inside each
# so we keep preprocessing here focused on defining the analysis dataset.
X.head()
```

Out[7]:

	age	bmi	avg_hr	total_steps	mean_vector_mag	heavy_activity_events
0	29	22.758307	84.719383	13625	42.201145	4
1	27	28.367524	74.532023	12415	37.210992	3
2	34	23.120624	77.538084	17822	40.162562	3
3	27	23.456790	71.918081	11958	37.652392	4
4	25	20.824656	74.884696	8129	26.695282	5

Model Estimation & Selection

We will compare an Ordinary Least Squares (OLS) multiple regression model against a Lasso regularized regression model. We also include a Naive Mean Baseline (Dummy Regressor) to evaluate if our models learn meaningful patterns beyond simply guessing the average sleep score.

To prevent data leakage, we utilize nested cross-validation and sci-kit learn `Pipeline` objects. All feature standardization (scaling) occurs independently within each of the 5 training folds. For the Lasso model, the optimal regularization penalty (alpha) is also selected via a grid search strictly within the training folds. This ensures our final Mean Squared Error reflects true predictive performance on entirely unseen data.

In [8]:
```
# Define X and y
feature_cols = ['age', 'bmi', 'avg_hr', 'total_steps', 'mean_vector_mag', 'heavy_ac
X = df[feature_cols].values
y = df['sleep_quality_score'].values

# Define the outer CV loop
kf = KFold(n_splits=5, shuffle=True, random_state=42)

print(f"Full dataset: {X.shape[0]} observations, {X.shape[1]} features")
```

```
Full dataset: 21 observations, 6 features
```

Model 1: OLS Multiple Linear Regression

Our first model is a baseline Ordinary Least Squares (OLS) regression using all 6 predictors. Feature scaling is handled internally via the pipeline during cross-validation to prevent data leakage.

In [9]:
```
ols_pipe = Pipeline([('scaler', StandardScaler()), ('model', LinearRegression()) ])

ols_fold_mse = -cross_val_score(ols_pipe, X, y, cv=kf, scoring='neg_mean_squared_er

for i, mse in enumerate(ols_fold_mse, 1):
    print(f" Fold {i} MSE: {mse:.2f}")
print(f"\nOLS Mean CV MSE : {ols_fold_mse.mean():.2f}")
print(f"OLS CV MSE Std  : {ols_fold_mse.std():.2f}")
```

```
Fold 1 MSE: 854.87
Fold 2 MSE: 159.16
Fold 3 MSE: 359.58
Fold 4 MSE: 146.13
Fold 5 MSE: 752.48

OLS Mean CV MSE : 454.45
OLS CV MSE Std  : 296.79
```

Model 2: Lasso Regression

Given our small sample size ($n = 21$) relative to the number of predictors ($p = 6$), a highly flexible model like OLS is severely prone to overfitting. Furthermore, our EDA revealed high multicollinearity between step count and vector magnitude.

To address this, we use Lasso Regression. Lasso is a less flexible model that penalizes the absolute size of the regression coefficients, effectively performing automatic feature selection by shrinking redundant or noisy variables to zero. To properly tune the regularization parameter (alpha) without leaking data from the test folds, we nest a `GridSearchCV` inside our cross-validation loop.

In [10]:
```
lasso_pipe = Pipeline([('scaler', StandardScaler()), ('model', Lasso(max_iter=10000

# Grid search for Lasso alpha
alpha_grid = np.logspace(-3, 0, 200)
lasso_search = GridSearchCV(
    estimator=lasso_pipe,
    param_grid={'model__alpha': alpha_grid},
    cv=5,
    scoring='neg_mean_squared_error'
)

lasso_fold_mse = -cross_val_score(lasso_search, X, y, cv=kf, scoring='neg_mean_squa

for i, mse in enumerate(lasso_fold_mse, 1):
    print(f" Fold {i} MSE: {mse:.2f}")
```

```
print(f"\nLasso Mean CV MSE : {lasso_fold_mse.mean():.2f}")
print(f"Lasso CV MSE Std  : {lasso_fold_mse.std():.2f}")
```

```
Fold 1 MSE: 784.99
Fold 2 MSE: 162.22
Fold 3 MSE: 153.95
Fold 4 MSE: 109.62
Fold 5 MSE: 596.12

Lasso Mean CV MSE : 361.38
Lasso CV MSE Std  : 275.90
```

Model Comparison Results

Model Complexity and Overfitting: The cross-validation results perfectly illustrate the dangers of overfitting in small datasets. The highly flexible OLS model exhibited the highest error (Mean CV MSE: 454.45), indicating that it memorized the training folds but generalized poorly to unseen data. The constrained Lasso model successfully reduced this complexity by shrinking redundant coefficients, improving the Mean CV MSE to 361.38.

However, it is critically important to note that the rigid Naive Mean Baseline achieved the lowest overall error (MSE: 293.83). This demonstrates that our 6 daytime features introduce more noise than predictive signal, meaning the patterns learned by OLS and Lasso were largely artifacts of overfitting rather than true underlying biological relationships.

In [11]:
```
# Naive Mean Predictor
dummy = DummyRegressor(strategy='mean')
dummy_fold_mse = -cross_val_score(dummy, X, y, cv=kf, scoring='neg_mean_squared_err

comparison = pd.DataFrame({
    'Model': ['OLS (baseline regression)', 'Lasso (L1 regression)', 'Naive mean bas
    'Mean CV MSE': [round(ols_fold_mse.mean(), 2), round(lasso_fold_mse.mean(), 2),
    'CV MSE Std': [round(ols_fold_mse.std(), 2), round(lasso_fold_mse.std(), 2), ro
})

print(comparison.to_string(index=False))
```

```
                   Model  Mean CV MSE  CV MSE Std
OLS (baseline regression)       454.45      296.79
    Lasso (L1 regression)       361.38      275.90
      Naive mean baseline       293.83      211.14
```

Coefficient Interpretation

Although the naive baseline outperformed the regression models in predictive accuracy, we still fit the best-performing regression model (Lasso) on the entire standardized dataset to estimate the final parameters. This allows us to interpret which features the algorithm weighed most heavily when attempting to find a signal.

The final standardized coefficient estimates (visualized in the bar chart below) indicate that `total_steps` and `avg_hr` held the strongest magnitudes. The L1 penalty successfully

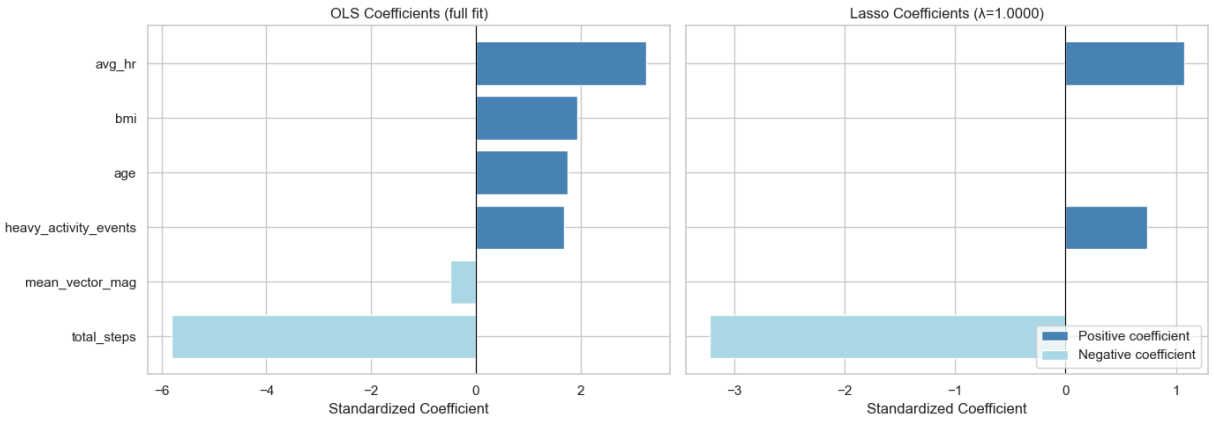
identified the severe multicollinearity between step count and vector magnitude, shrinking the `mean_vector_mag` coefficient completely to zero alongside `age`.

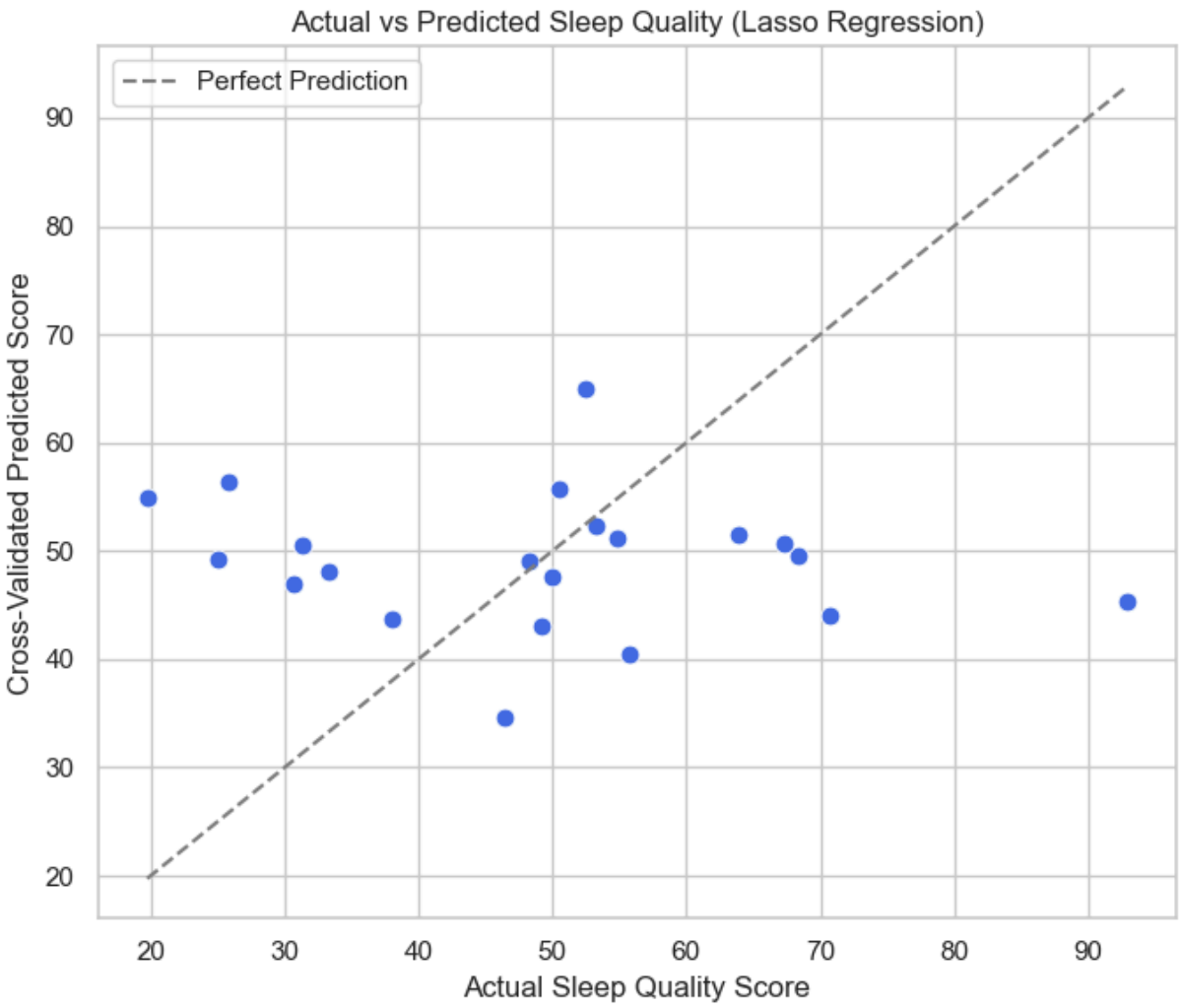
The scatter plot below visualizes the cross-validated predictions of this Lasso model against the actual sleep scores. The flat cluster of predictions compared to the wide spread of actual scores visually reinforces the high Mean Squared Error, illustrating the model's struggle to capture the true variance in the data.

In [12]:
```
# Fit models on the full dataset to extract final coefficients
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

ols = LinearRegression().fit(X_scaled, y)

# Find best alpha and automatically refit on full dataset
final_lasso_search = LassoCV(alphas=alpha_grid, cv=5, random_state=42, max_iter=100
final_lasso_search.fit(X_scaled, y)

# Create Coefficient DataFrame
coef_df = pd.DataFrame({
    'Feature': feature_cols,
    'OLS': ols.coef_,
    'Lasso': final_lasso_search.coef_
}).sort_values('OLS', ascending=True)

print(f"Final Lasso Parameter Estimates (Alpha = {final_lasso_search.alpha_:.4f}):"
print(coef_df[['Feature', 'Lasso']].to_string(index=False))

# Visualize Coefficients
fig, axes = plt.subplots(1, 2, figsize=(14, 5), sharey=True)
for ax, col, title in zip(axes, ['OLS', 'Lasso'],
                          ['OLS Coefficients (full fit)', f"Lasso Coefficients (\u0
    colors = ['steelblue' if v > 0 else 'lightblue' for v in coef_df[col]]
    ax.barh(coef_df['Feature'], coef_df[col], color=colors)
    ax.axvline(0, color='black', linewidth=0.8)
    ax.set_title(title)
    ax.set_xlabel('Standardized Coefficient')

from matplotlib.patches import Patch
legend_handles = [Patch(facecolor='steelblue', label='Positive coefficient'), Patch
axes[1].legend(handles=legend_handles, loc='lower right', frameon=True)
plt.tight_layout()
plt.show()

# Generate cross-validated predictions using the nested search from earlier
lasso_predictions = cross_val_predict(lasso_search, X, y, cv=kf)

plt.figure(figsize=(7, 6))
sns.scatterplot(x=y, y=lasso_predictions, color='royalblue', s=60)
mn = min(y.min(), lasso_predictions.min())
mx = max(y.max(), lasso_predictions.max())
plt.plot([mn, mx], [mn, mx], linestyle='--', color='gray', label='Perfect Predictio

plt.xlabel('Actual Sleep Quality Score')
```

```
plt.ylabel('Cross-Validated Predicted Score')
plt.title('Actual vs Predicted Sleep Quality (Lasso Regression)')
plt.legend()
plt.tight_layout()
plt.show()
```

```
Final Lasso Parameter Estimates (Alpha = 1.0000):
              Feature     Lasso
          total_steps -3.220543
      mean_vector_mag -0.000000
heavy_activity_events  0.744014
                  age -0.000000
                  bmi  0.000000
               avg_hr  1.076623
```





Discussion and Conclusion

Our initial hypothesis posited that higher levels of daytime physical activity (specifically total steps and heavy activity events) would positively predict a higher Sleep Quality Score. Based on our analysis, we fail to reject the null hypothesis.

The cross-validation results demonstrated that our predictive models were unable to find a generalizable signal, with the naive mean baseline generating a lower error rate than both OLS and Lasso regressions. Furthermore, in our final full-dataset estimation, the coefficient for `total_steps` presented a strong negative association (-3.22) rather than the hypothesized positive one. This suggests that simply moving more during the day does not inherently translate to a higher quality composite sleep score in this specific demographic.

Implications and Future Directions: A major limitation of this analysis is the small sample size ($n = 21$). Physiological data is highly noisy, and a sample of this size is insufficient for a machine learning model to separate genuine biological patterns from random variance without severe overfitting. For researchers interested in this topic, future steps must include acquiring a significantly larger and more diverse dataset. Additionally, rather than engineering a composite sleep score, future models might find more success attempting to

classify direct, objective clinical metrics, such as identifying the binary presence of severe sleep apnea directly from wearable heart rate variability.

In [14]:
```
!pip install nbconvert
!jupyter nbconvert --to html "COGS109_Final_Project.ipynb"
```

```
Requirement already satisfied: nbconvert in c:\users\antho\anaconda3\envs\cogs109\li
b\site-packages (7.17.0)
Requirement already satisfied: beautifulsoup4 in c:\users\antho\anaconda3\envs\cogs1
09\lib\site-packages (from nbconvert) (4.14.3)
Requirement already satisfied: bleach!=5.0.0 in c:\users\antho\anaconda3\envs\cogs10
9\lib\site-packages (from bleach[css]!=5.0.0->nbconvert) (6.3.0)
Requirement already satisfied: defusedxml in c:\users\antho\anaconda3\envs\cogs109\l
ib\site-packages (from nbconvert) (0.7.1)
Requirement already satisfied: jinja2>=3.0 in c:\users\antho\anaconda3\envs\cogs109
\lib\site-packages (from nbconvert) (3.1.6)
Requirement already satisfied: jupyter-core>=4.7 in c:\users\antho\anaconda3\envs\co
gs109\lib\site-packages (from nbconvert) (5.9.1)
Requirement already satisfied: jupyterlab-pygments in c:\users\antho\anaconda3\envs
\cogs109\lib\site-packages (from nbconvert) (0.3.0)
Requirement already satisfied: markupsafe>=2.0 in c:\users\antho\anaconda3\envs\cogs
109\lib\site-packages (from nbconvert) (3.0.3)
Requirement already satisfied: mistune<4,>=2.0.3 in c:\users\antho\anaconda3\envs\co
gs109\lib\site-packages (from nbconvert) (3.2.0)
Requirement already satisfied: nbclient>=0.5.0 in c:\users\antho\anaconda3\envs\cogs
109\lib\site-packages (from nbconvert) (0.10.4)
Requirement already satisfied: nbformat>=5.7 in c:\users\antho\anaconda3\envs\cogs10
9\lib\site-packages (from nbconvert) (5.10.4)
Requirement already satisfied: packaging in c:\users\antho\anaconda3\envs\cogs109\li
b\site-packages (from nbconvert) (26.0)
Requirement already satisfied: pandocfilters>=1.4.1 in c:\users\antho\anaconda3\envs
\cogs109\lib\site-packages (from nbconvert) (1.5.1)
Requirement already satisfied: pygments>=2.4.1 in c:\users\antho\anaconda3\envs\cogs
109\lib\site-packages (from nbconvert) (2.20.0)
Requirement already satisfied: traitlets>=5.1 in c:\users\antho\anaconda3\envs\cogs1
09\lib\site-packages (from nbconvert) (5.14.3)
Requirement already satisfied: typing-extensions in c:\users\antho\anaconda3\envs\co
gs109\lib\site-packages (from mistune<4,>=2.0.3->nbconvert) (4.15.0)
Requirement already satisfied: webencodings in c:\users\antho\anaconda3\envs\cogs109
\lib\site-packages (from bleach!=5.0.0->bleach[css]!=5.0.0->nbconvert) (0.5.1)
Requirement already satisfied: tinycss2<1.5,>=1.1.0 in c:\users\antho\anaconda3\envs
\cogs109\lib\site-packages (from bleach[css]!=5.0.0->nbconvert) (1.4.0)
Requirement already satisfied: platformdirs>=2.5 in c:\users\antho\anaconda3\envs\co
gs109\lib\site-packages (from jupyter-core>=4.7->nbconvert) (4.9.4)
Requirement already satisfied: jupyter-client>=6.1.12 in c:\users\antho\anaconda3\en
vs\cogs109\lib\site-packages (from nbclient>=0.5.0->nbconvert) (8.8.0)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\antho\anaconda3\en
vs\cogs109\lib\site-packages (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconver
t) (2.9.0.post0)
Requirement already satisfied: pyzmq>=25.0 in c:\users\antho\anaconda3\envs\cogs109
\lib\site-packages (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (27.1.
0)
Requirement already satisfied: tornado>=6.4.1 in c:\users\antho\anaconda3\envs\cogs1
09\lib\site-packages (from jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (6.5.
5)
Requirement already satisfied: fastjsonschema>=2.15 in c:\users\antho\anaconda3\envs
\cogs109\lib\site-packages (from nbformat>=5.7->nbconvert) (2.21.2)
Requirement already satisfied: jsonschema>=2.6 in c:\users\antho\anaconda3\envs\cogs
109\lib\site-packages (from nbformat>=5.7->nbconvert) (4.26.0)
Requirement already satisfied: attrs>=22.2.0 in c:\users\antho\anaconda3\envs\cogs10
9\lib\site-packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (26.1.0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in c:\users\anth
```

```
o\anaconda3\envs\cogs109\lib\site-packages (from jsonschema>=2.6->nbformat>=5.7->nbc
onvert) (2025.9.1)
Requirement already satisfied: referencing>=0.28.4 in c:\users\antho\anaconda3\envs
\cogs109\lib\site-packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (0.37.0)
Requirement already satisfied: rpds-py>=0.25.0 in c:\users\antho\anaconda3\envs\cogs
109\lib\site-packages (from jsonschema>=2.6->nbformat>=5.7->nbconvert) (0.30.0)
Requirement already satisfied: six>=1.5 in c:\users\antho\anaconda3\envs\cogs109\lib
\site-packages (from python-dateutil>=2.8.2->jupyter-client>=6.1.12->nbclient>=0.5.0
->nbconvert) (1.17.0)
Requirement already satisfied: soupsieve>=1.6.1 in c:\users\antho\anaconda3\envs\cog
s109\lib\site-packages (from beautifulsoup4->nbconvert) (2.8.3)
[NbConvertApp] Converting notebook COGS109_Final_Project.ipynb to html
[NbConvertApp] WARNING | Alternative text is missing on 5 image(s).
[NbConvertApp] Writing 637270 bytes to COGS109_Final_Project.html
```

In [ ]: